notoriously difficult to work with, and fibers (lightweight threads) force you to think in terms of other low-level constructs like await, interrupt, and join.

Concurrency is a means to achieve scalability: the premise is that, if needed, more CPUs can be added to servers, which the application then automatically starts using. It's the next best thing to a free lunch. *Scalability* is how a system adapts to a change in demand for resources without negatively affecting performance.

Unlike *scaling up* (increasing resources: for example, CPUs on existing servers), *scaling out* refers to dynamically adding more servers to a cluster. Not much has changed since the 1990s in terms of how programming languages support networking—many technologies still use remote procedure calls (RPCs) to communicate over the network.

In the meantime, advances in cloud computing services and multicore CPU architecture have made computing resources ever more abundant. Platform as a Service (PaaS) offerings have simplified the provisioning and deployment of very large distributed applications, once the domain of only the largest players in the IT industry. Cloud services like Amazon Web Services Elastic Compute Cloud (AWS EC2) and Google Compute Engine allow you to spin up thousands of servers in minutes, while tools like Docker and Kubernetes make it easier to package and manage applications on virtual servers.

The number of CPU cores in devices is also ever-increasing: even mobile phones and tablets have multiple CPU cores today. But that doesn't mean you can afford to throw any number of resources at any problem. In the end, everything is about cost and efficiency: effectively scaling applications to get more bang for your buck. Just